Chapter 5 ▪ Exceptions, Assertions, and Logging

In many programs, dealing with the unexpected can be more complex than implementing the "happy day" scenarios. Like most modern programming languages, Java has a robust exception-handling mechanism for transferring control from the point of failure to a competent handler. In addition, the `assert` statement provides a structured and efficient way of expressing internal assumptions. Finally, you will see how to use the logging API to keep a record of the various events, be they routine or suspicious, in the execution of your programs.

The key points of this chapter are:

1. When you throw an exception, control is transferred to the nearest handler of the exception.
2. In Java, checked exceptions are tracked by the compiler.
3. Use the `try` / `catch` construct to handle exceptions.
4. The try-with-resources statement automatically closes resources after normal execution or when an exception occurred.
5. Use the `try` / `finally` construct to deal with other actions that must occur whether or not execution proceeded normally.
6. You can catch and rethrow an exception, or chain it to another exception.
7. A stack trace describes all method calls that are pending at a point of execution.
8. An assertion checks a condition, provided that assertion checking is enabled for the class, and throws an error if the condition is not fulfilled.
9. The platform logging API is a simple logging frontend that is a part of the JDK.
10. The `java.util.logging` package provides a simple logging backend.

5.1. Exception Handling

What should a method do when it encounters a situation in which it cannot fulfill its contract? The traditional answer was that the method should return some error code. But that is cumbersome for the programmer calling the method. The caller is obliged to check for errors, and if it can't handle them, return an error code to its own caller. Not

unsurprisingly, programmers didn't always check and propagate return codes, and errors went undetected, causing havoc later.

Instead of having error codes bubble up the chain of method calls, Java supports *exception handling* where a method can signal a serious problem by "throwing" an exception. One of the methods in the call chain, but not necessarily the direct caller, is responsible for handling the exception by "catching" it. The fundamental advantage of exception handling is that it decouples the processes of detecting and handling errors. In the following sections, you will see how to work with exceptions in Java.

5.1.1. Throwing Exceptions

A method may find itself in a situation where it cannot carry out the task at hand. Perhaps a required resource is missing, or it was called with inconsistent arguments. In such a case, it is best to throw an exception.

Suppose you implement a method that yields a random integer between two bounds:

```
public static int randInt(int low, int high) {
   return low + (int) (Math.random() * (high - low + 1));
}
```

What should happen if someone calls `randInt(10, 5)`? Trying to fix this is probably not a good idea because the caller might have been confused in more than one way. Instead, throw an appropriate exception:

```
if (low > high)
   throw new IllegalArgumentException(
      "low should be <= high but low is %d and high is %d".formatted(low, high));
```

As you can see, the `throw` statement is used to "throw" an object of a class—here, `IllegalArgumentException`. The object is constructed with a debugging message. You will see in the next section how to pick an appropriate exception class.

When a `throw` statement executes, the normal flow of execution is interrupted immediately. The `randInt` method stops executing and does not return a value to its caller. Instead, control is transferred to a handler, as you will see in **Section 5.1.4**.

5.1.2. The Exception Hierarchy

Figure 5.1 shows the hierarchy of exceptions in Java. All exceptions are subclasses of the class `Throwable`. Subclasses of `Error` are exceptions that are thrown when something exceptional happens that the program cannot be expected to handle, such as memory exhaustion. There is not much you can do about errors other than giving a message to the user that things have gone very wrong.

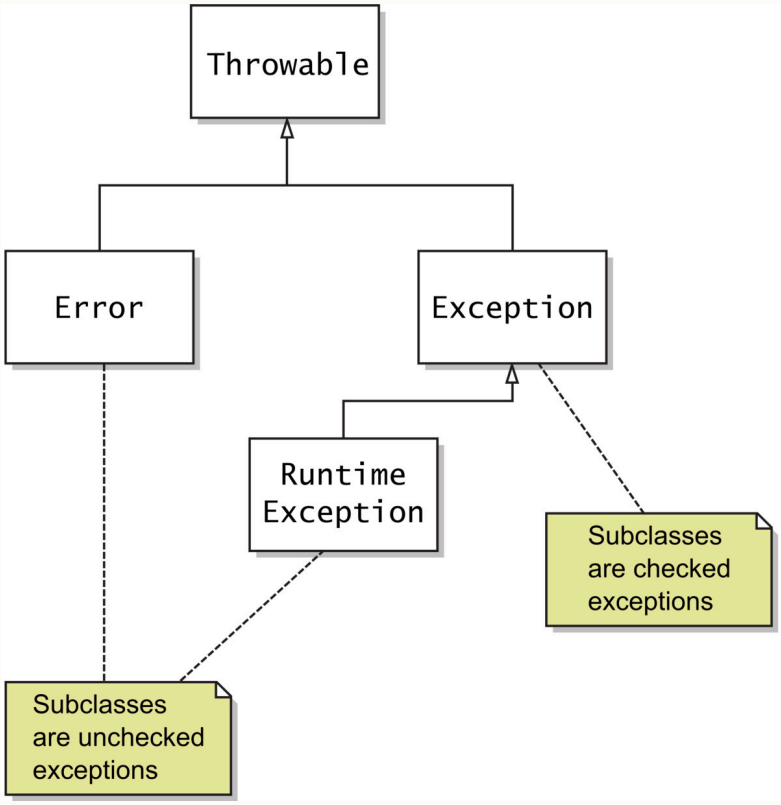


Figure 5.1: The exception hierarchy

Programmer-reported exceptions are subclasses of the class `Exception`. These exceptions fall into two categories:

- *Unchecked* exceptions are subclasses of `RuntimeException`.
- All other exceptions are *checked* exceptions.

As you will see in the next section, programmers must either catch checked exceptions or declare them in the method header. The compiler checks that these exceptions are handled properly.



Note: The name `RuntimeException` is unfortunate. Of course, all exceptions occur at runtime. However, the exceptions that are subclasses of `RuntimeException` are not checked during compilation.

Checked exceptions are used in situations where failure should be anticipated. One common reason for failure is input and output. Files may be damaged, and network connections may fail. A number of exception classes extend `IOException`, and you should use an appropriate one to report any errors that you encounter. For example, when a file that should be there turns out not be, throw a `FileNotFoundException`.

Unchecked exceptions indicate logic errors caused by programmers, not by unavoidable external risks. For example, a `NullPointerException` is not checked. Just about any method might throw one, and programmers shouldn't spend time on catching them. Instead, they should make sure that no `null`s are dereferenced in the first place.

Sometimes, implementors need to use their judgment to make a distinction between checked and unchecked exceptions. Consider the call `Integer.parseInt(str)`. It throws an unchecked `NumberFormatException` when `str` doesn't contain a valid integer. On the other hand, `Class.forName(str)` throws a checked `ClassNotFoundException` when `str` doesn't contain a valid class name.

Why the difference? The reason is that it is possible to check whether a string is a valid integer before calling `Integer.parseInt`, but it is not possible to know whether a class can be loaded until you actually try to load it.

The Java API provides many exception classes, such as `IOException`, `IllegalArgumentException`, and so on. You should use these when appropriate. However, if none of the standard exception classes is suitable for your purpose, you can create your own by extending `Exception`, `RuntimeException`, or another existing exception class.

When you do so, it is a good idea to supply both a no-argument constructor and a constructor with a message string. For example,

```
public class FileFormatException extends IOException {
   public FileFormatException() {}
   public FileFormatException(String message) {
      super(message);
   }
   // Also add constructors for chained exceptions—see Section 5.1.7
}
```

5.1.3. Declaring Checked Exceptions

Any method that might give rise to a checked exception must declare it in the method header with a `throws` clause:

```
public void write(Object obj, String filename)
   throws IOException, ReflectiveOperationException
```

List the exceptions that the method might throw, either because of a `throw` statement or because it calls another method with a `throws` clause.

In the `throws` clause, you can combine exceptions into a common superclass. Whether or not that is a good idea depends on the exceptions. For example, if a method can throw multiple subclasses of `IOException`, it makes sense to cover them all in a clause `throws IOException`. But if the exceptions are unrelated, don't combine them into `throws Exception`—that would defeat the purpose of exception checking.



Tip: Some programmers think it is shameful to admit that a method might throw an exception. Wouldn't it be better to handle it instead? Actually, the opposite is true. You should allow each exception to find its way to a competent handler. The golden rule of exceptions is, "Throw early, catch late."

When you override a method, it cannot throw more checked exceptions than those declared by the superclass method. For example, if you extend the `write` method from the beginning of this section, the overriding method can throw fewer exceptions:

```
public void write(Object obj, String filename)
   throws FileNotFoundException
```

But if the method tried to throw an unrelated checked exception, such as an `InterruptedException`, it would not compile.



Caution: If the superclass method has no `throws` clause, then no overriding method can throw a checked exception.

You can use the javadoc `@throws` tag to document when a method throws a (checked or unchecked) exception. Most programmers only do this when there is something meaningful to document. For example, there is little value in telling users that an

`IOException` is thrown when there is a problem with input/output. But comments such as the following can be meaningful:

```
@throws NullPointerException if filename is null
@throws FileNotFoundException if there is no file with name filename
```



Note: You never specify the exception type of a lambda expression. However, if a lambda expression can throw a checked exception, you can only pass it to a functional interface whose method declares that exception. For example, the call

```
list.forEach(obj -> write(obj, "output.dat"));
```

is an error. The parameter type of the `forEach` method is the functional interface

```
public interface Consumer<T> {
   void accept(T t);
}
```

The `accept` method is declared not to throw any checked exception.

5.1.4. Catching Exceptions

To catch an exception, set up a `try` block. In its simplest form, it looks like this:

```
try {
   statements
} catch (ExceptionClass ex) {
   handler
}
```

If an exception of the given class occurs as the statements in the `try` block are executed, control transfers to the handler. The exception variable (`ex` in our example) refers to the exception object which the handler can inspect if desired.

There are two modifications that you can make to this basic structure. You can have multiple handlers for different exception classes:

```
try {
   statements
} catch (ExceptionClass1 ex) {
   handler1
} catch (ExceptionClass2 ex) {
   handler2
} catch (ExceptionClass3 ex) {
   handler3
}
```

The `catch` clauses are matched top to bottom, so the most specific exception classes must come first.

Alternatively, you can share one handler among multiple exception classes:

```
try {
   statements
} catch (ExceptionClass1 | ExceptionClass2 | ExceptionClass3 ex) {
   handler
}
```

In that case, the handler can only call those methods on the exception variable that belong to all exception classes.

5.1.5. The Try-with-Resources Statement

One problem with exception handling is resource management. Suppose you write to a file and close it when you are done:

```
ArrayList<String> lines = ...;
var out = new PrintWriter("output.txt");
for (String line : lines) {
   out.println(line.toLowerCase());
}
out.close();
```

This code has a hidden danger. If any method throws an exception, the call to `out.close()` never happens. That is bad. Output could be lost, or if the exception is triggered many times, the system could run out of file handles.

A special form of the `try` statement can solve this issue. You can specify *resources* in the header of the `try` statement. A resource must belong to a class implementing the `AutoCloseable` interface. You can declare variables in the `try` block header:

```
ArrayList<String> lines = ...;
try (var out = new PrintWriter("output.txt")) { // Variable declaration
   for (String line : lines)
      out.println(line.toLowerCase());
}
```

Alternatively, you can provide previously declared effectively final variables in the header:

```
var out = new PrintWriter("output.txt");
try (out) { // Effectively final variable
   for (String line : lines)
      out.println(line.toLowerCase());
}
```

The `AutoCloseable` interface has a single method

```
public void close() throws Exception
```



Note: There is also a `Closeable` interface. It is a subinterface of `AutoCloseable`, also with a single `close` method. However, that method is declared to throw an `IOException`.

When the `try` block exits, either because its end is reached normally or because an exception is thrown, the `close` methods of the resource objects are invoked. For example:

```
try (var out = new PrintWriter("output.txt")) {
   for (String line : lines) {
      out.println(line.toLowerCase());
   }
} // out.close() called here
```

You can declare multiple resources, separated by semicolons. Here is an example with two resource declarations:

```
try (var in = new Scanner(Path.of("/usr/share/dict/words"));
      var out = new PrintWriter("output.txt")) {
   while (in.hasNext())
      out.println(in.next().toLowerCase());
}
```

The resources are closed in reverse order of their initialization—that is, `out.close()` is called before `in.close()`.

Suppose that the `PrintWriter` constructor throws an exception. Now `in` is already initialized but `out` is not. The `try` statement does the right thing: calls `in.close()` and propagates the exception.

Some `close` methods can throw exceptions. If that happens when the `try` block completed normally, the exception is thrown to the caller. However, if another exception had been thrown, causing the `close` methods of the resources to be called, and one of them throws an exception, that exception is likely to be of lesser importance than the original one.

In this situation, the original exception gets rethrown, and the exceptions from calling `close` are caught and attached as "suppressed" exceptions. This is a very useful mechanism that would be tedious to implement by hand (see **Exercise 10**). When you catch the primary exception, you can retrieve the secondary exceptions by calling the `getSuppressed` method:

```
try {
   ...
} catch (IOException ex) {
   Throwable[] secondaryExceptions = ex.getSuppressed();
   ...
}
```

If you want to implement such a mechanism yourself in a (hopefully rare) situation when you can't use the try-with-resources statement, call `ex.addSuppressed(secondaryException)`.

A try-with-resources statement can optionally have `catch` clauses that catch any exceptions in the statement.

5.1.6. The `finally` Clause

As you have seen, the try-with-resources statement automatically closes resources whether or not an exception occurs. Sometimes, you need to clean up something that isn't an `AutoCloseable`. In that case, use the `finally` clause:

```
try {
   Do work
} finally {
   Clean up
}
```

The `finally` clause is executed when the `try` block comes to an end, either normally or due to an exception.

This pattern occurs whenever you need to acquire and release a lock, or increment and decrement a counter, or push something on a stack and pop it off when you are done. You want to make sure that these actions happen regardless of what exceptions might be thrown.

You should avoid throwing an exception in the `finally` clause. If the body of the `try` block was terminated due to an exception, it is masked by an exception in the `finally` clause. The suppression mechanism that you saw in the preceding section only works for try-with-resources statements.

Similarly, a `finally` clause should not contain a `return` statement. If the body of the `try` block also has a `return` statement, the one in the `finally` clause replaces the return value. Consider this example:

```
public static int parseInt(String s) {
   try {
      return Integer.parseInt(s);
   } finally {
      return 0; // Error
   }
}
```

It looks as if in the call `parseInt("42")`, the body of the `try` block returns the integer 42. However, the `finally` clause is executed before the method actually returns and causes the method to return 0, ignoring the original return value.

And it gets worse. Consider the call `parseInt("zero")`. The `Integer.parseInt` method throws a `NumberFormatException`. Then the `finally` clause is executed, and the `return` statement swallows the exception!



Tip: The body of the finally clause is intended for cleaning up resources. Don't put statements that change the control flow (`return`, `throw`, `break`, `continue`) inside a `finally` clause.

It is possible to form `try` statements with `catch` clauses followed by a `finally` clause. But you have to be careful with exceptions in the `finally` clause. For example, have a look at this `try` block adapted from an online tutorial:

```
BufferedReader in = null;
try {
   in = Files.newBufferedReader(path);
   Read from in
} catch (IOException ex) {
   System.err.println("Caught IOException: " + ex.getMessage());
} finally {
   if (in != null) {
      in.close(); // Caution—might throw an exception
   }
}
```

The programmer clearly thought about the case when the `Files.newBufferedReader` method throws an exception. It appears as if this code would catch and print all I/O exceptions, but it actually misses one: the one that might be thrown by `in.close()`. It is often better to rewrite a complex `try`/`catch`/`finally` statement as a try-with-resources statement or by nesting a `try`/`finally` inside a `try`/`catch` statement—see **Exercise 12**.

5.1.7. Rethrowing and Chaining Exceptions

When an exception occurs, you may not know what to do about it, but you may want to log the failure. In that case, rethrow the exception so that a competent handler can deal with it:

```
try {
   Do work
}
catch (Exception ex) {
   logger.log(level, message, ex);
   throw ex;
}
```



Note: Something subtle is going on when this code is inside a method that may throw a checked exception. Suppose the enclosing method is declared as

```
public void read(String filename) throws IOException
```

At first glance, it looks as if one would need to change the `throws` clause to `throws Exception`. However, the Java compiler carefully tracks the flow and realizes that `ex` could only have been an exception thrown by one of the statements in the `try` block, not an arbitrary `Exception`.

Sometimes, you want to change the class of a thrown exception. For example, you may need to report a failure of a subsystem with an exception class that makes sense to the user of the subsystem. Suppose you encounter a database error in a servlet. The code that executes the servlet may not want to know in detail what went wrong, but it definitely wants to know that the servlet is at fault. In this case, catch the original exception and chain it to a higher-level one:

```
try {
   Access the database
}
catch (SQLException ex) {
   throw new ServletException("database error", ex);
}
```

When the `ServletException` is caught, the original exception can be retrieved as follows:

```
Throwable cause = ex.getCause();
```

The `ServletException` class has a constructor that takes as a parameter the cause of the exception. Not all exception classes do that. In that case, you have to call the `initCause` method, like this:

```
try {
   Access the database
}
catch (SQLException ex) {
   var ex2 = new CruftyOldException("database error");
   ex2.initCause(ex);
   throw ex2;
}
```

If you provide your own exception class, you should provide, in addition to the two constructors described in **Section 5.1.2**, the following constructors:

```
public class FileFormatException extends IOException {
   ...
   public FileFormatException(Throwable cause) { initCause(cause); }
   public FileFormatException(String message, Throwable cause) {
      super(message);
      initCause(cause);
   }
}
```



Tip: The chaining technique is also useful if a checked exception occurs in a method that is not allowed to throw a checked exception. You can catch the checked exception and chain it to an unchecked one.

5.1.8. Uncaught Exceptions and the Stack Trace

If an exception is not caught anywhere, a *stack trace* is displayed—a listing of all pending method calls at the point where the exception was thrown. The stack trace is sent to `System.err`, the stream for error messages.

If you want to save the exception somewhere else, perhaps for inspection by your tech support staff, set the default uncaught exception handler:

```
Thread.setDefaultUncaughtExceptionHandler((thread, ex) -> {
   Record the exception
});
```



Note: An uncaught exception terminates the thread in which it occurred. If your application only has one thread (which is the case for the programs that you have seen so far), the program exits after invoking the uncaught exception handler.

Sometimes, you are forced to catch an exception and don't really know what to do with it. For example, the `Class.forName` method throws a checked exception that you need to handle. Instead of ignoring the exception, at least print the stack trace:

```
try {
   Class<?> cl = Class.forName(className);
   ...
} catch (ClassNotFoundException ex) {
   ex.printStackTrace();
}
```

If you want to store the stack trace of an exception, you can put it into a string as follows:

```
var out = new ByteArrayOutputStream();
ex.printStackTrace(new PrintWriter(out));
String description = out.toString();
```



Note: If you need to process the stack trace in more detail, use the `StackWalker` class. For example, the following prints all stack frames:

```
StackWalker walker = StackWalker.getInstance();
walker.forEach(frame -> System.err.println("Frame: " + frame));
```

You can also analyze the `StackWalker.StackFrame` instances in detail. See the API documentation for details.

5.1.9. API Methods for Throwing Exceptions

The `Objects` class has a method for convenient null checks of parameters. Here is a sample usage:

```
public void process(String direction) {
   this.direction = Objects.requireNonNull(direction);
   ...
}
```

If `direction` is `null`, a `NullPointerException` is thrown. At first glance, this seems a bit pointless since a `NullPointerException` would eventually be thrown when the value is used. But consider working back from a stack trace. When you see a call to `requireNonNull` as the culprit, you know right away what you did wrong.

You can also supply a message string for the exception:

```
this.direction = Objects.requireNonNull(direction, "direction must not be null");
```

A variant of this method allows you to supply an alternate value instead of throwing an exception:

```
this.direction = Objects.requireNonNullElse(direction, "North");
```

To compute the alternate value only when needed, use the following variant:

```
this.directory = Objects.requireNonNullElseGet(directory,
   () -> System.getProperty("user.dir"));
```

The lambda expression is only evaluated if `directory` is `null`.

There is also a convenience method for making a range check:

```
this.index = Objects.checkIndex(index, length);
```

The method returns the index if it is between 0 and `length` - 1, and throws an `IndexOutOfBoundsException` otherwise.

5.2. Assertions

Assertions are a commonly used idiom of defensive programming. Suppose you are convinced that a particular property is fulfilled, and you rely on that property in your code. For example, you may be computing

```
double y = Math.sqrt(x);
```

You are certain that `x` is not negative. Still, you want to double-check rather than have "not a number" floating-point values creep into your computation. You could, of course, throw an exception:

```
if (x < 0) throw new IllegalStateException(x + " < 0");
```

But this condition stays in the program, even after testing is complete, slowing it down. The assertion mechanism allows you to put in checks during testing and to have them automatically removed in the production code.



Note: In Java, assertions are intended as a debugging aid for validating internal assumptions, not as a mechanism for enforcing contracts. For example, if you want to report an inappropriate argument of a public method, don't use an assertion but throw an `IllegalArgumentException`.

5.2.1. Using Assertions

There are two forms of the assertion statement in Java:

```
assert condition;
assert condition : expression;
```

The `assert` statement evaluates the condition and throws an `AssertionError` if it is false. In the second form, the expression is turned into a string that becomes the message of the error object.



Note: If the expression is a `Throwable`, it is also set as the cause of the assertion error (see Section 5.1.7).

For example, to assert that `x` is non-negative, you can simply use the statement

```
assert x >= 0;
```

Or you can pass the actual value of `x` into the `AssertionError` object so it gets displayed later:

```
assert x >= 0 : x;
```

5.2.2. Enabling and Disabling Assertions

By default, assertions are disabled. Enable them by running the program with the `-enableassertions` or `-ea` option:

```
java -ea MainClass
```

You do not have to recompile your program because enabling or disabling assertions is handled by the class loader. When assertions are disabled, the class loader strips out the assertion code so that it won't slow execution. You can even enable assertions in specific classes or in entire packages, for example:

```
java -ea:MyClass -ea:com.mycompany.mylib... MainClass
```

This command turns on assertions for the class `MyClass` and all classes in the `com.mycompany.mylib` package *and its subpackages*. The option `-ea...` turns on assertions in all classes of the default package.

You can also disable assertions in certain classes and packages with the `-disableassertions` or `-da` option:

```
java -ea:... -da:MyClass MainClass
```

When you use the `-ea` and `-da` switches to enable or disable all assertions (and not just specific classes or packages), they do not apply to the "system classes" that are loaded

without class loaders. Use the `-enablesystemassertions` switch (or the short form `-esa`) to enable assertions in system classes.

It is also possible to programmatically control the assertion status of class loaders with the following methods:

```
void ClassLoader.setDefaultAssertionStatus(boolean enabled);
void ClassLoader.setClassAssertionStatus(String className, boolean enabled);
void ClassLoader.setPackageAssertionStatus(String packageName, boolean enabled);
```

As with the `-enableassertions` command-line option, the `setPackageAssertionStatus` method sets the assertion status for the given package and its subpackages.

5.3. Logging

Every Java programmer is familiar with the process of inserting `System.out.println` calls into troublesome code to gain insight into program behavior. Of course, once you have figured out the cause of trouble, you remove the print statements—only to put them back in when the next problem surfaces. Logging frameworks are designed to overcome this problem.

5.3.1. Should You Use the Java Logging Framework?

Java has a standard logging framework, usually called after its package name `java.util.logging`, and sometimes abbreviated as `j.u.l`. However, other logging frameworks have more features and are in common use, such as Log4j (**https://logging.apache.org/log4j/2.x**) and Logback (**https://logback.qos.ch**).

If you want to give users of your code the choice of logging framework, then you should use a "façade" library that sends log messages to the preferred framework. A commonly used façade with a pleasant API is SLF4J (**https://www.slf4j.org**). Another façade is the "platform logging API" (also known as JEP 246). It is very basic but a part of the JDK. The façade is sometimes called the *frontend*. It provides the API that programmers uses to log messages. The *backend* is in charge of filtering and formatting the messages, and putting them somewhere. The backend needs to be configurable by deployers, usually by editing configuration files.

In the following sections, I will show you how to use the platform logging API as front-end and `java.util.logging` as a backend. This can be a reasonable choice if you find the

frontend API sufficient, since you can always swap out the backend.

The `java.util.logging` backend has fewer features than its more popular alternatives, but it suffices for many use cases. Because of its simplicity, it is less susceptible to attacks. In contrast, obscure features of Log4j allowed hackers to craft program inputs that, when logged, caused malicious code execution.

Whether or not you end up using them, studying the platform logging API and `java.util.logging` backend gives you a good foundation of the capabilities of logging frameworks.

5.3.2. Logging 101

Platform loggers implement the `System.Logger` interface. Each logger has a name. The name can be arbitrary, but it is often the package name of the class whose methods generate logging messages. You get a platform logger like this:

```
System.Logger logger = System.getLogger("com.mycompany.myapp");
```

When you request a logger with a given name for the first time, it is created. Subsequent calls to the same name yield the same logger object.

Now you are ready to log:

```
logger.log(System.Logger.Level.INFO, "Opening file " + filename);
```

The record is printed like this:

```
Aug 04, 2022 09:53:34 AM com.mycompany.myapp read
INFO: Opening file data.txt
```

Note that the time and the names of the calling class and method are automatically included.

To turn off these informational messages when your program is deployed, you can configure the backend. In the case of the `java.util.logging` backend, prepare a file `logging.properties` with the following contents:

```
handlers=java.util.logging.ConsoleHandler
com.mycompany.myapp.level=WARNING
```

Then start the application like this:

```
java -Djava.util.logging.config.file=logging.properties com.mycompany.myapp.Main
```

Since the `INFO` level is below the `WARNING` level, the message no longer shows up.

The API for getting the logger and logging a message are part of the frontend—in this case, the platform logging API. If you use a different frontend, the API will be different.

The message destination, formatting, and filtering, as well as the mechanisms for the configuration are part of the backend—here, `java.util.logging`. If you use a different backend, follow its instructions for configuration.

5.3.3. The Platform Logging API

As you saw in the preceding section, each logged message has a level. The enumeration `System.Logger.Level` has the following values, in decreasing severity: `ERROR`, `WARNING`, `INFO`, `DEBUG`, and `TRACE`.



Tip: With the import statement

```
import static java.lang.System.Logger.Level.*;
```

you can shorten the levels:

```
logger.log(INFO, "Opening file " + filename);
   // Instead of System.Logger.Level.INFO
```

In the above example, the message `"Opening file " + filename` is created even if the message is suppressed. If you are concerned about the cost of creating the message string, you can use a lambda expression instead:

```
logger.log(INFO, () -> "Opening file " + filename);
```

Then the message is only computed when it is actually logged.

It is common to log an exception:

```
catch (IOException ex) {
   logger.log(WARNING, "Cannot open file " + filename, ex);
}
```

You can format the message using a pattern:

```
logger.log(WARNING, "Cannot open file {0}", filename);
```



Caution: This is *not* a `printf` style pattern. The pattern is processed by the `MessageFormat` class that is used for localization of program messages. See Volume II for details. Here, you simply need to know that string placeholders are `{0}`, `{1}`, and so on. Braces must be escaped with single quotes: `'{'`, `'}'`. Use two single quotes `''` to include a literal single quote in the message.

Log messages can be localized to different languages, using the resource bundle mechanism that is explained in the JavaDoc of the `ResourceBundle` class.

Supply the bundle and the key for the formatting string:

```
logger.log(WARNING, bundle, "file.bad", filename);
   // Looks up file.bad in the bundle
```

Alternatively, get the logger as

```
System.Logger logger = System.getLogger("com.mycompany.myapp", bundle);
```

When calling one of the logger methods with a `String` parameter for the message or format, the argument is interpreted as a key in the bundle. The methods with `Object` or `Supplier<String>` parameters are unaffected.

Some, but not all combinations of these features (deferred message computation, adding a throwable, formatting, using a bundle), are supported. **Table 5.1** shows the

complete API.

Table 5.1: The `System.Logger` API

Method	Description
`String getName()`	The name of this logger.
`boolean isLoggable(System.Logger.Level level)`	`true` if this logger processes logs at the given level.
`void log(System.Logger.Level level, String msg)`	Logs the given message.
`void log(System.Logger.Level level, Object obj)`	Logs the message `obj.toString()` .
`void log(System.Logger.Level level, String msg, Throwable thrown)`	Logs the given message and throwable.
`void log(System.Logger.Level level, Supplier<String> msgSupplier)`	If this logger processes logs at the given level, invokes the supplier and logs the result.

Method	Description
`void log(System.Logger.Level level, Supplier<String> msgSupplier, Throwable thrown)`	If this logger processes logs at the given level, invokes the supplier and logs the result and throwable.
`void log(System.Logger.Level level, String format, Object... params)`	Logs the formatted message with the given parameters.
`void log(System.Logger.Level level, ResourceBundle bundle, String msg, Throwable thrown)`	Logs the message in the bundle with key `msg` (or, if `bundle` is `null`, `msg` itself) and the throwable.
`void log(System.Logger.Level level, ResourceBundle bundle, String format, Object... params)`	Looks up the format with key `format` in the bundle (or, if `bundle` is `null`, uses `format` itself), and logs the formatted message with the given parameters.

5.3.4. Logging Configuration

Let us now turn to the logging backend. As already mentioned, the default backend of the platform logging API is `java.util.logging`. The information in the following sections is specific to that backend.

You can change various properties of the backend by editing a configuration file. The default configuration file is located at `conf/logging.properties` in the JDK. To use another file, set the `java.util.logging.config.file` property to the file location by starting your application with

```
java -Djava.util.logging.config.file=configFile MainClass
```



Caution: Calling `System.setProperty("java.util.logging.config.file", configFile)` in `main` has no effect because the log manager is initialized during VM startup, before `main` executes.

You can specify the logging levels for your own loggers by adding lines such as

```
com.mycompany.myapp.level=WARNING
```

That is, append the `.level` suffix to the logger name.



Note: Properties in the logging configuration are not system properties. You cannot set them with the `-D` command-line option. Instead, place the logging properties into a file that you specify with `-Djava.util.logging.config.file=configFile`.

You can also specify the root level:

```
.level=WARNING
```



Caution: For historical reasons, some of the levels have different names in the platform logging API and the `java.util.logging` framework. **Table 5.2** shows the correspondences.

Table 5.2: Corresponding Levels for Platform Logging and the Java Logging Framework

Platform logging	`java.util.logging`
`ERROR`	`SEVERE`
WARNING	WARNING
`INFO`	`INFO`
DEBUG	FINE

Platform logging	java.util.logging
TRACE	FINER

Similar to package names, logger names are hierarchical. In fact, they are more hierarchical than packages. There is no semantic relationship between a package and its parent, but logger parents and children share certain properties. For example, if you turn off messages to the logger `"com.mycompany"`, then its child loggers are also deactivated.

As you will see in the next section, loggers don't actually send the messages to the console—that is the job of the handlers. Handlers also have levels. To see `DEBUG` / `FINE` messages on the console, you also need to set

```
java.util.logging.ConsoleHandler.level=FINE
```



Caution: The settings in the log manager configuration are not system properties. Starting a program with `-Dcom.mycompany.myapp.level=FINE` does not have any effect on the logger.

It is also possible to change logging levels in a running program by using the `jconsole` program. For details, see **https://www.oracle.com/technetwork/articles/java/jconsole-1564139.html#LoggingControl**.

5.3.5. Log Handlers

The backend of the `java.util.logging` API is based on *handlers*. The simplest handler is the `ConsoleHandler` that prints log records to the `System.err` stream.

Each handler has a parent. By default, the handler simply sends each record to the parent handler. The standard logging configuration uses a `ConsoleHandler` by default. Its default logging level is `INFO`.

To send log records elsewhere, add another handler. The `java.util.logging` API provides two handlers for this purpose: a `FileHandler` and a `SocketHandler`. The `SocketHandler` sends records to a specified host and port. Of greater interest is the `FileHandler` that collects records in a file.

To add a file handler, provide this entry in the logging properties:

```
handlers=java.util.logging.ConsoleHandler,java.util.logging.FileHandler
```

The records are sent to a file `javan.log` in the user's home directory, where n is a number to make the file unique. By default, the records are formatted in XML. A typical log record has the form

```
<record>
  <date>2014-08-04T09:53:34</date>
  <millis>1407146014072</millis>
  <sequence>1</sequence>
  <logger>com.mycompany.myapp</logger>
  <level>INFO</level>
  <class>com.horstmann.corejava.Employee</class>
  <method>read</method>
  <thread>10</thread>
  <message>Opening file staff.txt</message>
</record>
```

You can modify the default behavior of the file handler by setting various parameters in the log manager configuration (see **Table 5.3**).

You probably don't want to use the default log file name. Use a pattern such as `%h/myapp.log` (see **Table 5.4** for an explanation of the pattern variables).

If multiple applications (or multiple copies of the same application) use the same log file, you should turn the `append` flag on. Alternatively, use `%u` in the file name pattern so that each application creates a unique copy of the log.

It is also a good idea to turn file rotation on. Log files are kept in a rotation sequence, such as `myapp.log.0`, `myapp.log.1`, `myapp.log.2`, and so on. Whenever a file exceeds the size limit, the oldest log is deleted, the other files are renamed, and a new file with generation number `0` is created.

Table 5.3: File Handler Configuration Parameters

Configuration property	Description	Default

Configuration property	Description	Default
java.util.logging.FileHandler.level	The handler level.	Level.ALL
java.util.logging.FileHandler.append	When true, log records are appended to an existing file; otherwise, a new file is opened for each program run.	false
java.util.logging.FileHandler.limit	The approximate maximum number of bytes to write in a file before opening another (0 = no limit).	0 in the FileHandler class, 50000 in the default log manager configuration
java.util.logging.FileHandler.pattern	The file name pattern (see **Table 5.4**).	%h/java%u.log
java.util.logging.FileHandler.count	The number of logs in a rotation sequence.	1 (no rotation)

Configuration property	Description	Default
java.util.logging.FileHandler.filter	The filter for filtering log records (see Section 5.3.6).	No filtering
java.util.logging.FileHandler.encoding	The character encoding.	The platform character encoding
java.util.logging.FileHandler.formatter	The formatter for each log record.	java.util.logging.XMLFormatter

Table 5.4: Log File Pattern Variables

Variable	Description
%h	The user's home directory (the user.home property).
%t	The system's temporary directory.
%u	A unique number.
%g	The generation number for rotated logs. A .%g suffix is used if rotation is specified and the pattern doesn't contain %g .
%%	The percent character.

5.3.6. Filters and Formatters

Besides filtering by logging levels, each handler can have an additional filter that implements the Filter interface, a functional interface with a method

```
boolean isLoggable(LogRecord record)
```

To install a filter into a handler, add an entry such as the following into the logging configuration:

```
java.util.logging.ConsoleHandler.filter=com.mycompany.myapp.MyFilter
```

The `ConsoleHandler` and `FileHandler` classes emit the log records in text and XML formats. However, you can define your own formats as well. Extend the `java.util.logging.Formatter` class and override the method

```
String format(LogRecord record)
```

Format the record in any way you like and return the resulting string. In your `format` method, you can get information about the `LogRecord` by calling one of the methods in **Table 5.5**.

In your `format` method, you may want to call the method

```
String formatMessage(LogRecord record)
```

That method formats the message part of the record, looking up the message key in a resource bundle and substituting message format parameters.

Many file formats (such as XML) require head and tail parts that surround the formatted records. To achieve this, override the methods

```
String getHead(Handler h)
String getTail(Handler h)
```

Finally, set the formatter in the logging configuration:

```
java.util.logging.FileHandler.formatter=com.mycompany.myapp.MyFormatter
```

Table 5.5: `LogRecord` Property Getters

Method	Property

Method	Property
`Level getLevel()`	The logging level of this record.
String getLoggerName()	The name of the logger that is logging this record.
`ResourceBundle getResourceBundle()` `String getResourceBundleName()`	The resource bundle, or its name, to be used for localizing the message, or `null` if none is provided.
String getMessage()	The “raw” message before localization or formatting.
`Object[] getParameters()`	The parameter objects, or `null` if none is provided.
Throwable getThrown()	The thrown object, or null if none is provided.
`String getSourceClassName()` `String getSourceMethodName()`	The location of the code that logged this record. This information may be supplied by the logging code or automatically inferred from the runtime stack. It might be inaccurate if the logging code supplied the wrong value or if the running code was optimized so that the exact location cannot be inferred.
Instant getInstant()	The creation time.
`long getSequenceNumber()`	The unique sequence number of this record.
long getLongThreadID()	The unique ID for the thread in which this record was created. These IDs are assigned by the LogRecord class and have no relationship to other thread IDs.

Exercises

1. Write a method `public ArrayList<Double> readValues(String filename) throws ...` that reads a file containing floating-point numbers. Throw appropriate exceptions if the file could not be opened or if some of the inputs are not floating-point numbers.
2. Write a method `public double sumOfValues(String filename) throws ...` that calls the preceding method and returns the sum of the values in the file. Propagate any exceptions to the caller.
3. Write a program that calls the preceding method and prints the result. Catch the exceptions and provide feedback to the user about any error conditions.
4. Repeat the preceding exercise, but don't use exceptions. Instead, have `readValues` and `sumOfValues` return error codes of some kind.
5. Implement a method that contains the code with a `Scanner` and a `PrintWriter` in **Section 5.1.5**. But don't use the try-with-resources statement. Instead, just use `catch` clauses. Be sure to close both objects, provided they have been properly constructed. You need to consider the following conditions:
   1. The `Scanner` constructor throws an exception.
   2. The `PrintWriter` constructor throws an exception.
   3. `hasNext`, `next`, or `println` throw an exception.
   4. `out.close()` throws an exception.
   5. `in.close()` throws an exception.
6. **Section 5.1.6** has an example of a broken `try` statement with `catch` and `finally` clauses. Fix the code with (a) catching the exception in the `finally` clause, (b) a `try`/`catch` statement containing a `try`/`finally` statement, and (c) a try-with-resources statement with a `catch` clause.
7. Explain why

```
try (var in = new Scanner(Path.of("/usr/share/dict/words"));
      var out = new PrintWriter(outputFile)) {
   while (in.hasNext())
      out.println(in.next().toLowerCase());
}
```

is better than

```
var in = new Scanner(Path.of("/usr/share/dict/words"));
var out = new PrintWriter(outputFile);
try (in; out) {
```

```
    while (in.hasNext())
      out.println(in.next().toLowerCase());
  }
```

8. For this exercise, you'll need to read through the source code of the `java.util.Scanner` class. If input fails when using a `Scanner`, the `Scanner` class catches the input exception and closes the resource from which it consumes input. What happens if closing the resource throws an exception? How does this implementation interact with the handling of suppressed exceptions in the try-with-resources statement?
9. Design a helper method so that one can use a `ReentrantLock` in a try-with-resources statement. Call `lock` and return an `AutoCloseable` whose `close` method calls `unlock` and throws no exceptions.
10. The methods of the `Scanner` and `PrintWriter` classes do not throw checked exceptions to make them easier to use for beginning programmers. How do you find out whether errors occurred during reading or writing? Note that the constructors *can* throw checked exceptions. Why does that defeat the goal of making the classes easier to use for beginners?
11. Write a recursive `factorial` method in which you print all stack frames before you return the value. Construct (but don't throw) an exception object of any kind and get its stack trace, as described in **Section 5.1.8**.
12. Compare the use of `Objects.requireNonNull(obj)` and `assert obj != null`. Give a compelling use for each.
13. Write a method `int min(int[] values)` that, just before returning the smallest value, asserts that it is indeed ≤ all values in the array. Use a private helper method or, if you already peeked into **Chapter 8**, `Stream.allMatch`. Call the method repeatedly on a large array and measure the runtime with assertions enabled, disabled, and removed.
14. Implement and test a log record filter that filters out log records containing bad words such as sex, drugs, and C++.
15. Implement and test a log record formatter that produces a JSON file.